ETP : A Custom Transport Layer Protocol

Creator : RaspiestSheep3
Project Type: Transport Layer Protocol
Project Name : Emergency Transfer Protocol (ETP)
Project Link : https://github.com/RaspiestSheep3/Custom_Networking_Protocol

1 - Summary

This RFC proposes a custom transport layer protocol I name Emergency Transfer Protocol (ETP) as an alternative to existing protocols such as TCP, UDP and QUIC. This protocol is designed to be lossless and effective in extreme network conditions, such as areas impacted by natural disasters. An outline of design choices is given, and the protocol is simulated in C++.

2 - Problem Statement

The majority of modern applications rely on one of 3 transport layer protocols - TCP, UDP or QUIC. Whilst these protocols are undeniably effective, each face issues, especially when used in extreme network conditions (note that in this report extreme networks refer to networks with high congestion, high latency, high packet loss rate and/or low bandwidth):

- **TCP:** If a packet is lost, all subsequent pacers are paused despite their validity (Head-of-Line Blocking), which can heavily impact efficiency on slow networks. Moreover, if a packet is lost TCP assumes this is a congestion issue and slows down data transmission and therefore throughput.
- **UDP:** Because there is no insurance that all packets are being received on a network with high packet loss a large amount of packets will be dropped and never resent. This means key systems such as downloading files and messaging are unusable on extreme systems, and even applications such as image downloads become very low-quality.
- **QUIC:** Whilst it performs better than TCP and UDP on extreme networks, QUIC still faces some flaws. Primarily, QUIC encrypts the data before transmission, which can make it inefficient on the CPU for encrypting and decrypting data. QUIC is also based on UDP, which means that it is often blocked by firewalls as a byproduct of shutting down UDP-based malware attacks.

These protocols therefore do not perform effectively in systems without good network coverage. In particular, areas affected by natural disasters or war suffer infrastructure damage, taking down many system connections. This then leads to the network suffering high latency and packet loss, which can severely limit traditional communication systems.

3 - Proposed Solution

3.1 - Project Proposal

The goal for this project is to design a custom transport-layer protocol named Emergency Transport Protocol (ETP) able to form packets able to be transferred between machines. The protocol will then be implemented in C++ and tested by simulating key factors such as packet loss, latency and bandwidth.

ETP is designed for operation in extreme network creations such as those caused by heavy infrastructure damage, where traditional protocols are weak.

3.2 - Design Specifications

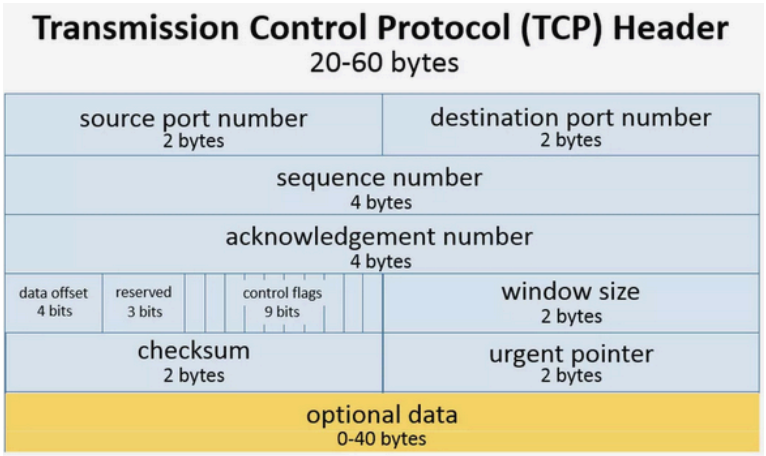
The protocol should match some key specification points:

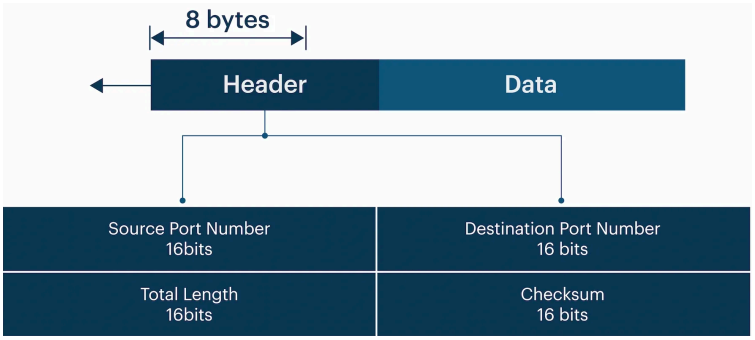
- **Lossless data transmission (primary)** - any data transmitted using the protocol should be guaranteed to arrive at the destination, even at extreme packet loss rate. This is the main flaw with UDP, and as such it should be corrected in this project
- **High throughput under extreme conditions (primary)** - the protocol should be efficient under extreme conditions, as this is the major flaw with TCP
- **Not built on top of existing protocols (primary)** - the protocol should not be layered over an existing transport layer protocol, as this is the major flaw with QUIC
- **Lightweight (secondary)** - the protocol should not be very intensive on the CPU or GPU, as this was a flaw with QUIC
- **Easily implementable (secondary)** - the protocol should be easy to implement within the standard OSI model

4 - Protocol Design

4.1 - Packet Structure

When operating over standard IPv6 Ethernet, the Maximum Transmission Unit (MTU) is 1500B, with the IPv6 header using 40B - this leaves us with 1460B for the ETP header and payload. For a highly efficient system the packet header should be as small as possible so more payload data can be transmitted per packet; this then reduces the number of packets on the network so data transmission is more effective.





We may first include 2 bytes each for the sender and recipient port number, as this is required for data to be properly transmitted between different users. I then choose to implement a sequence number - because ETP is designed to be a lossless protocol the receiver needs some way of identifying which packets have not been received so they can be re-requested, and using a sequence number as opposed to a different identifier means that the recipient can easily reorder packages upon receiving them Using 4B for the sequence number means we can handle 2^{31} packets per session (the first bit is used to signify sender vs receiver) - this should be more than enough for all modern communication systems. Following this, 1 byte is dedicated to flags - individual bits can be toggled on and off in order to compactly convey packet information. After this, we generate a checksum of the entire packet, which is stored using 2B. Finally, 1B is used for the ETP version - this will be treated as 0x00 in this report as this is the first version.. Together these form a header of 12B, which is 1.5xthe size of UDP and nearly half the minimum size of TCP. Notably, because the OPTIONS section of TCP has been eliminated, we have significantly decreased the maximum header packet size significantly, and increased network reliability, as every packet follows the exact same structure.

ETP Packet Structure - Overall Packet is 1460B, Header is 12B, MSS is 1448B

Source Port Number (2B)	Destination Port Number (2B)	
Sequence Number (4B)		
Checksum (2B)	Flags (1B)	Version (1B)
Payload (1448B)		

Source Port Number: The port the package has been sent from
Destination Port Number: The port the package is being sent to
Sequence Number: The ordered number of the packet during a communication session
Flags: Individual bits are used to mark different properties of the packet
Checksum: A hash of the packet used to check packet integrity
Version: The protocol version being used - 0x00 is used for this report
Payload: The data being transmitted by the packet + padding

4.1.1 - Header Flags

Bit Position	Flag Code	Flag Description
08 **MSB**	END	Last packet within packet mini-set
07	RSV	Reserved for forward compatibility
06	STA	First packet of a packet set. The first 4 bytes contain the last intended

		packet sequence number. If ACK = 1 acknowledgement of packet request
05	FIN	Request to gracefully close connection - if ACK = 0 step 1 of closing handshake, else step 2 of closing handshake.
04	URG	Packet should be processed as soon as possible and forwarded up the OSI model immediately upon receipt.
03	RES	Request to resend packets listed in payload - ACK = 1 if RES = 1. If RES = 0 but the payload is empty, all packets were correctly received.
02	ACK	Acknowledgement of packet delivery. Often combined with other flags.
01 **LSB**	SYN	Sent from the client to the server to initialise a connection - if ACK = 0, SYN1. Else, SYN1-ACK

4.2 - Initial Handshake

ETP uses a 2-way handshake, in a similar manner to TCP's 3-way handshake. The main difference is that TCP first establishes a connection and then later sends data, which adds an extra packet to network transmission. In order to solve this, the client first sends a SYN1 packet to the server. The server then responds with a SYN1-ACK. When using TCP, the client then sends a 3rd SYN2, which then fully initialises the connection. However, by making the client respond immediately with data we can cut out the SYN2; the main consequence of this change is that the connection should be started with the data already prepared for transit. However, this is a minor cost for the benefit of reducing the number of unnecessary packets per communication.

Within the SYN1 payload, 2 key pieces of information are transmitted within the message payload. The window size describes how much information each machine can buffer at a time, to stop overloading which would lead to packets being dropped. A max round trip time (RTT) is also added, which is defined by applications higher up in the OSI model. If responses are not received within this time frame, it is assumed a packet is lost. Whilst this will inevitably lead to some packets being unnecessarily transmitted multiple times, this will be an issue regardless of what the chosen trip time is, so making this a parameter open to the higher layers means this value can be adjusted to suit the network it's being deployed on and the application using it. A sequence number of 0 is used The rest of the packet is padded and the message length is set appropriately, which allows for the full formation of the ETP SYN1 packet.

ETP SYN1 Packet

<Source Port Number (2B)>	*<Destination Port Number (2B)>*	
0x00000000		
<Checksum (2B)>	0x01	0x00
<Max Window Size (4B)> <RTT> (1B)>		

The SYN1-ACK packet mostly mirrors the SYN1 packet. However, the 3 main key differences are the sequence number, the flags and the payload - the sequence number has the first bit as 1, the ACK flag is set as active and the RTT is not sent as the client defines this parameter.

ETP SYN1-ACK Packet

<Source Port Number (2B)>	*<Destination Port Number (2B)>*

0x80000000		
<Checksum (2B)>	0x03	0x00
<Max Window Size (4B)>		

When initialising the connection, the client sends the SYN1 packet, receives the SYN1-ACK packet and then immediately sends the first data packet. If either machine does not receive the expected packet within the set RTT the packet will be assumed to be lost and the packet will be resent.

4.3 - Subsequent Data Transfer

Once the handshake has occurred, data can then be transmitted in packets. Each transmission can be thought of as a packet set. The first packet in the packet set is referred to as the *STA* packet, and the first 4 bytes of the payload contain the sequence number of the last packet in the packet set. All subsequent packets hold the message data, with the last packet being padded if need be. When the sender sends the *STA* packet, the recipient will then respond with a *STA-ACK* packet, which confirms that the recipient is ready for transmission. This is done because if the recipient is not ready there is no point in flooding the network with extra packets. If the *STA-ACK* packet is not received within the RTT the *STA* packet is resent.

ETP STA Packet

<Source Port Number (2B)>	*<Destination Port Number (2B)>*	
<Sender's Sequence Number (4B)>		
<Checksum (2B)>	0x20	0x00
<Sender's Sequence Number of Final Packet (4B)>		

ETP STA-ACK Packet

<Source Port Number (2B)>	*<Destination Port Number (2B)>*	
<Recipient's Sequence Number (4B)>		
<Checksum (2B)>	0x22	0x00
<STA-ACK Payload Hexadecimal Code(1B)>		

STA-ACK Payload Hexadecimal Code	Meaning
0x00	Transfer accepted - proceed with data transfer
0x01	Transfer declined - manual decline
0x02	Transfer declined - recipient is overwhelmed
0x03 onwards	Transfer declined - custom reasons - these can be set by higher layers or specific applications

If the recipient declines the message, a 3 way handshake is used to convey this info. In step 1, the recipient sends the *STA-ACK* packet, and keeps sending it until it receives a *STA-END* packet. In step 2,

the sender sends a *STA-END* packet upon receiving a *STA-ACK* packet, and keeps sending until it gets a *STA-END-ACK* packet. In step 3, upon receiving a *STA-END* packet the recipient sends a *STA-END-ACK* packet, confirming the data is not being transferred.

ETP STA-END Packet

<Source Port Number (2B)>	<Destination Port Number (2B)>	
<Sender's Sequence Number (4B)>		
<Checksum (2B)>	0xA0	0x00

ETP STA-END-ACK Packet

<Source Port Number (2B)>	<Destination Port Number (2B)>	
<Recipient's Sequence Number (4B)>		
<Checksum (2B)>	0xA2	0x00

Like with the initial handshake, as soon as *STA-ACK* is received data is transmitted assuming an agreement is made. Packets from the packet-set are sent up to the minimum of the max window size and the number of sequence numbers that can be fit in the payload - this is a mini-set. Any URG packets should be sent before all other packets. The last packet within the packet set will have the END flag in order to signify this is the end of the packet mini-set. At the end of each packet-mini set the recipient will then send a *RES* packet describing which - if any - packets are missing. If the sender does not receive a *RES* within the RTT after sending the *END* packet of the mini-set, it will resend only the *END* packet of the mini-set. Similarly, if the recipient does not receive the new packets, it will resend the *RES* packet. Once a *RES* packet has been sent with 0 requested packets, the next mini-set will be transmitted.

ETP Standard Packet

<Source Port Number (2B)>	<Destination Port Number (2B)>	
<Sender's Sequence Number (4B)>		
<Checksum (2B)>	0x00	0x00
<Payload (up to 1448B)>		

ETP URG Packet

<Source Port Number (2B)>	<Destination Port Number (2B)>	
<Sender's Sequence Number (4B)>		
<Checksum (2B)>	0x08	0x00
<Payload (up to 1448B)>		

ETP END Packet

<Source Port Number (2B)>	<Destination Port Number (2B)>	
<Sender's Sequence Number (4B)>		
<Checksum (2B)>	0x80	0x00

<Payload (up to 1448B)>		

ETP RES Packet

<Source Port Number (2B)>	*<Destination Port Number (2B)>*	
<Recipient's Sequence Number (4B)>		
<Checksum (2B)>	0x05	0x00
<Missing Packet Sequence Numbers (up to 1448B)>		

Once all packets within the packet-set have been transmitted and the final *RES* has 0 requested packets, a 3-way handshake is used. In step 1, the recipient sends the *RES* with 0 packets as just mentioned. In step 2, the sender sends an *END* packet with no data. In step 3, the recipient sends an *END-ACK*. This handshake follows the TCP model: If step 1 fails the recipient will keep sending *RES* until it gets an *END*, if the *END* fails it will be resent upon receiving a *RES* packet, and if the *END-ACK* is not received by the client it will resend the *END* until it gets *END-ACK*.

ETP END-ACK Packet

<Source Port Number (2B)>	*<Destination Port Number (2B)>*	
<Recipient's Sequence Number (4B)>		
<Checksum (2B)>	0x82	0x00

4.4 - Session Closing

5 - Protocol Testing

6 - Conclusion